

Student Operating System

Complete End-to-End Architecture, Workflow Specification & Jev Integration Blueprint

Scope: A build-ready conceptual and technical architecture for a school-wide operating system that manages the student lifecycle from curriculum design through teaching, assessment, evaluation, observation, intervention, progression, reporting and institutional learning.

Core thesis: the school becomes a continuously learning system. The persistent object is the student state; the school knowledge graph defines what should be learned; workflows determine what should happen; AI generates and interprets; Jev supplies bounded decisions; deterministic software enforces policy; teachers retain approval authority.

Prepared 26 September 2026

1. Executive Architecture Decision

Build this as a Student Operating System, not an LMS. An LMS primarily stores content and assignments. This system maintains a longitudinal, evidence-backed model of each child and continuously uses it to determine what should happen next.

The OS should have six foundational layers: (1) School Knowledge Graph, (2) Content & Pedagogy Layer, (3) Student State Graph, (4) Decision Fabric including Jev, (5) Workflow/Orchestration Layer, and (6) Human Interfaces & Governance.

Jev's role: a bounded decision substrate. It should classify, score, choose, verify and route. It should not be the source of truth, the database, the workflow engine, or the primary content generator.

Current TypeSafe documentation describes Jev/System One as an API for structured Choice, Score and Noul (yes/no) decisions. TypeSafe's current API documentation shows a shared state plus up to 32 named questions per request.

■cite■turn0search6■

The architecture below deliberately keeps Jev behind a provider-neutral Decision Service. That means the OS can use Jev heavily without making Jev an architectural single point of failure.

2. Master Architecture

Layer 1 — School Knowledge Graph

Standards → subjects → topics → skills → learning outcomes → mastery levels → prerequisites → misconceptions → evidence types → pedagogy patterns → assessments

↓

Layer 2 — Curriculum & Content System

Pedagogy generation → lesson/activity design → question bank → rubrics → validated intervention library

↓

Layer 3 — Student State Graph

Mastery + evidence + misconceptions + observations + interventions + trajectory + confidence + history

↓

Layer 4 — Decision Fabric

Jev + deterministic rules + statistical models + specialist AI + confidence calibration

↓

Layer 5 — Workflow Orchestrator

Events → state transitions → tasks → approvals → actions → new evidence

↓

Layer 6 — Human Interfaces

Teacher OS • Coordinator OS • Curriculum OS • Parent reports • Student experience • Admin

Cross-cutting: identity/permissions, audit trail, observability, model registry, prompt/decision versioning, privacy, evaluation, data governance.

3. What Is the Brain? What Is the Nervous System?

The brain is not a model. The brain is the combination of the School Knowledge Graph + Student State Graph + Policy/Workflow Engine.

School Knowledge Graph = institutional memory. It defines what the school believes should be learned and how concepts relate.

Student State Graph = current memory of the child. It records evidence, mastery estimates, misconceptions, observations, interventions and trajectories.

Decision Fabric = interpretation. Jev and other models turn evidence into bounded judgments.

Orchestrator = executive function. It converts decisions into workflow transitions and tasks.

Teacher = constitutional authority. Humans approve consequential curriculum, assessment and developmental decisions.

Event Store = memory of what actually happened. It allows the system to replay, audit and learn from every action.

So when we say Jev can become the 'nervous system', we mean it can sit at thousands of small decision points between state and action. It senses/assesses, produces structured signals, and triggers the appropriate workflow. It does not become the school itself.

4. Knowledge Graph Architecture

The Knowledge Graph is the stable semantic backbone. A relational database alone is insufficient because the most important relationships are graph-like: prerequisite-of, measures, misconception-of, taught-by, assessed-by, demonstrated-by, similar-to and leads-to.

Node	Examples
Standard	NCERT/CBSE/IGCSE requirement
Subject	Mathematics
Topic	Subtraction
Skill	3-digit subtraction
Learning outcome	Subtract with regrouping
Mastery level	Beginning / Developing / Secure / Advanced
Prerequisite	Place value
Misconception	Fails to decrement source place
Pedagogy	Concrete blocks / number line
Activity	Subtraction Market
Question	Q-2941
Rubric	Regrouping rubric
Intervention	Visual regrouping activity
Evidence	Assessment response / observation
Student state	Student-specific mastery node

Recommended implementation: PostgreSQL as transactional source of truth + graph capability (e.g. Neo4j or a graph layer) for relationship-heavy queries. Do not force every school record into the graph. Use the graph where relationships are the product.

5. Student State Graph

Every student should have a machine-readable longitudinal state. It must be versioned and evidence-backed.

Student

→ Subject

→ Skill

→ Mastery estimate

→ Evidence

→ Error patterns

→ Teacher observations

→ Interventions

→ Response to interventions

→ Trajectory

Field	Meaning
mastery	Current estimate, not a permanent label
confidence	Confidence in that estimate
evidence_count	Number/type of supporting observations
last_evidence	Recency
trajectory	Improving / stable / declining / uncertain
misconceptions	Observed error patterns with evidence
intervention_history	What was tried and what happened
teacher_observations	Structured qualitative evidence
next_action	Current recommended workflow state
provenance	Source, timestamp, model and version

Qualitative traits should be time-dependent and evidence-weighted. Avoid permanent labels such as 'anxious' or 'lazy' based on isolated observations.

6. Event-Driven Core

The OS should be event-driven. Every meaningful action produces an immutable event; workflows subscribe to events and create the next state transition.

Examples

- TopicCompleted
- LessonDelivered
- TeacherObservationCreated
- AssessmentGenerated
- AssessmentApproved
- AssessmentSubmitted
- AssessmentEvaluated
- ErrorPatternDetected
- MasteryUpdated
- InterventionRecommended
- InterventionApproved
- InterventionCompleted
- ReassessmentCompleted
- ReportDrafted
- ReportApproved

Each event should carry student_id, actor, timestamp, source, object IDs, workflow ID, decision IDs, model/prompt versions where relevant, confidence and approval status.

7. Workflow Catalog

Workflow	Trigger	Core output
W01 Curriculum Design	Skill map created/changed	Approved curriculum
W02 Pedagogy Generation	Learning outcome exists	Validated lesson/activity
W03 Curriculum QA	Content generated/changed	QA findings
W04 Question Bank	Skill/level defined	Validated questions
W05 Assessment Assembly	Topic completed / due	Student-specific assessment
W06 Assessment Approval	Assessment generated	Teacher-approved assessment
W07 Submission Ingestion	Paper/image received	Structured evidence
W08 Evaluation	Evidence available	Score + error analysis
W09 Mastery Update	Evaluation completed	Updated student state
W10 Intervention	Mastery threshold crossed	Recommended intervention
W11 Reassessment	Intervention completed	New evidence
W12 Observation	Teacher note/voice uploaded	Structured observations
W13 Teacher Daily OS	Day begins / events arrive	Prioritized tasks
W14 Timetable/Planning	Schedule state changes	Daily/weekly plan
W15 Handover	Student/teacher changes	Context brief
W16 Reporting	Period ends	Draft reports
W17 Curriculum Learning	Aggregate evidence available	Curriculum improvement proposals
W18 Institutional Learning	Cross-school evidence	System-level improvements

8. W01–W03: Curriculum & Pedagogy Workflows

W01 — Curriculum design

Input: standards + grade + subject + master skill map. Output: versioned learning outcomes, levels, prerequisites and curriculum sequence. Human: curriculum team.

W02 — Pedagogy generation

Input: skill + outcome + level + pedagogy philosophy + constraints. LLM generates activity, materials, teacher moves, observation points, student outcomes, theory alignment, differentiation and assessment opportunity.

Jev checks: alignment, age appropriateness, prerequisite sufficiency, target-skill coverage, misconception coverage, assessability and implementation risk.

W03 — Curriculum QA

AI-generated content is independently evaluated before human approval. Store both generated version and approved version. Curriculum team edits/approves; approval locks a version.

9. W04–W09: Assessment → Evaluation → Mastery

W04 Question bank: every question is mapped to skill, level, misconception, evidence type, rubric and prerequisite.

W05 Assessment assembly: deterministic filters narrow the pool; Jev selects/weights diagnostically useful items; code enforces coverage, length, difficulty and exposure constraints.

W06 Approval: teacher sees assessment, rationale and relevant student state, then approves/edits/rejects.

W07 Submission: scan/image → page detection → question segmentation → handwriting/OCR/multimodal extraction.

W08 Evaluation: deterministic scoring where possible; symbolic math where possible; multimodal model for working; Jev for bounded error classification, evidence sufficiency and escalation.

W09 Mastery: combine objective evidence with statistical/probabilistic update. Jev can provide interpretation signals, but the canonical mastery state should be maintained by your own state engine.

10. W12 Observation Engine

Teacher can submit structured form or voice note. Speech-to-text creates transcript. Extraction layer identifies student references, academic observations, behaviour observations, engagement, social interactions, curriculum/pedagogy feedback and operational issues.

Jev then evaluates bounded questions such as: Was a specific skill mentioned? Was a misconception mentioned? Is there evidence of transfer difficulty? Is this likely a recurring pattern? Should this enter persistent state? Does this need teacher/coordinator review?

Observation objects should preserve raw transcript/audio reference, extracted claims, evidence spans, confidence, author and timestamp. Never discard the source evidence behind a qualitative inference.

11. W10–W11 Intervention Loop

Detect: Student State indicates gap.

Diagnose: Jev classifies likely issue and evidence sufficiency.

Select: Jev chooses among validated interventions.

Approve: teacher confirms.

Execute: activity/lesson occurs.

Observe: teacher records qualitative response.

Reassess: targeted assessment.

Update: mastery and intervention-effectiveness evidence stored.

Next: continue / intensify / change intervention / progress.

This loop should become the core adaptive mechanism of the school.

12. Teacher OS

The Teacher OS should be a thin operational interface over the underlying system. The teacher should not navigate the architecture.

Screen	What it shows
Today	Timetable, lessons, tasks, alerts, approvals
Class	Student status, current skills, attention signals
Student	Longitudinal state, evidence, strengths, gaps, interventions
Assessments	Generated, pending approval, completed, review-needed
Observations	Voice/text input and structured history
Recommendations	Why the system recommends an action
Curriculum feedback	Teacher feedback on activity/pedagogy
Handover	Concise evidence-backed student brief
Reports	Draft report + evidence + approval

The system should minimize clicks. Voice input should be first-class for observations and teacher feedback.

13. Agentic Platform: What Should We Use?

Recommendation: use a durable workflow/state-machine architecture as the core; use agent frameworks only inside bounded workflows. Do not make a free-form multi-agent swarm the backbone of a school system.

LangGraph: a strong candidate for workflows requiring explicit state, branching, persistence and human interruption. Its current human-in-the-loop documentation describes interrupting execution, persisting graph state, and resuming after approve/edit/reject/respond decisions. ■cite■turn0search8■

OpenAI Agents SDK: also viable for code-first agent workflows. Current documentation supports tools, handoffs, guardrails, human review, resumable state and tracing. ■cite■turn0search2■turn0search7■

My recommendation: if your team is building a Python/TypeScript product and wants maximum explicit control, use **LangGraph for long-running, stateful workflows** or the OpenAI Agents SDK where agent loops/handoffs are genuinely useful. Do not use both as co-equal orchestration layers in v1. Pick one workflow runtime.

Jev should sit below either framework as a Decision Service. The workflow engine owns state transitions; Jev owns bounded judgments.

14. Recommended Technical Stack

Layer	Recommendation	Reason
Frontend	Next.js / React	Teacher/coordinator interfaces
API	TypeScript/NestJS or Python/FastAPI	Typed service boundaries
Transactional DB	PostgreSQL	Canonical records + permissions
Workflow runtime	LangGraph OR OpenAI Agents SDK	Stateful workflows
Event bus	Temporal / Kafka / managed queue as scale demands	Reliable asynchronous events
Knowledge graph	Neo4j or PostgreSQL graph extension	Relationship-heavy knowledge
Object storage	S3-compatible	Scans, audio, images
Vector retrieval	pgvector initially	Simple semantic retrieval
LLM	Provider abstraction; frontier model(s)	Generation/reasoning
Decision model	Jev via Decision Service	Bounded choices/scores
Speech	STT provider	Teacher voice notes
Vision	OCR/HMER/multimodal model	Assessment ingestion
Observability	OpenTelemetry + workflow traces	Debug/evaluation
Analytics	Warehouse later; Postgres initially	Institutional analytics
Auth	School SSO/RBAC	Role separation

15. Jev Decision Fabric

Create one internal service called **Decision Service**. Every Jev call goes through it.

```
DecisionRequest
decision_id
state_snapshot
evidence_refs
context
policy_version
requested_confidence_profile
trace_id

DecisionResult
decision_id
choice / score / noul
probabilities
confidence
model
model_version
decision_spec_version
evidence_refs
latency
cost
fallback_used
timestamp
```

The Decision Service should support caching, idempotency, retries, fallback models, thresholding, logging, evaluation replay and feature flags. Current System One documentation supports client-generated idempotency keys, typed questions and shared state; use those capabilities where appropriate. ■cite■turn0search6■

16. Decision Registry

Every AI decision must be a versioned product object.

Field	Example
decision_id	EVAL.ERROR_CLASSIFICATION.V1
purpose	Classify primary mathematical misconception
question_type	choice
allowed_outputs	regrouping, place_value, arithmetic, procedure, transcription, unknown
input_schema	question + expected + answer + working + student context
policy	confidence $\geq$ X $\rightarrow$ auto; else teacher
fallback	frontier multimodal model / manual
owner	Assessment team
golden_dataset	dataset://eval-error-v3
model_version	jev-latest / pinned version
prompt/spec version	decision-spec-14
metrics	accuracy, calibration, review rate
status	shadow / advisory / autonomous

17. How the System Keeps Improving

The OS needs multiple learning loops, not one generic 'AI improves' loop.

Loop	Feedback	Improvement
Child loop	Assessment → intervention → reassessment	Personal trajectory
Teacher loop	Observation → outcome → feedback	Better teacher support
Pedagogy loop	Activity → mastery change	Better activities
Curriculum loop	Aggregate outcomes → curriculum review	Better curriculum
Decision loop	Jev output → human correction	Better decision calibration
Model loop	Golden dataset → eval → model/prompt change	Better AI
Institution loop	Across cohorts/classes	Better school operations

Human corrections are not merely overrides. They should become labelled data. A teacher correcting 'regrouping error' from Jev should create an evaluation example for that Decision ID.

18. Model / Workflow Improvement Flywheel

Production event → decision made → teacher accepts/edits/rejects → outcome observed → disagreement captured → example enters evaluation set → decision spec/prompt/model tested → offline evaluation → shadow deployment → online A/B or controlled rollout → new version → monitor → repeat

Do not let production automatically retrain or rewrite prompts. The improvement pipeline should be controlled, versioned and reversible.

19. Knowledge Graph + Student Graph + Decision Graph

Use three related but distinct graphs:

- **School Knowledge Graph:** what should be learned and how concepts relate.
- **Student State Graph:** what this child has demonstrated and what evidence supports it.
- **Decision/Evidence Graph:** why the system made a decision, what evidence it used, what human did, and what happened afterward.

This gives you explainability and longitudinal learning. A report statement can trace back to evidence; an intervention can trace to a diagnosis; a diagnosis can trace to assessment work; curriculum feedback can trace to aggregated student outcomes.

20. Reporting + Handover

Reports should be generated from state, not invented from scratch.

Pipeline: Student State → evidence selection → period comparison → growth analysis → narrative generation → Jev evidence verification → teacher approval → final report.

For handover, use the same mechanism: summarize only high-value evidence relevant to the next teacher's context. Include confidence, recency and what has already been tried.

21. Autonomy Ladder

Level	System behavior
0 — Record	AI observes; human decides
1 — Suggest	AI recommends; human approves
2 — Guarded automation	AI acts automatically only under explicit policy
3 — Adaptive workflow	System selects among approved pathways
4 — Closed-loop optimization	System continuously learns which pathways work
5 — Institutional intelligence	School-level curriculum/pedagogy evolves through governed evidence

The school should progress one decision class at a time. Do not attempt Level 4/5 autonomy before the evidence, event and governance infrastructure exists.

22. Governance, Safety & Privacy

- Role-based access: teacher, coordinator, curriculum team, principal, parent, student, admin.
- Every sensitive action is auditable.
- Separate raw evidence from derived inference.
- Retention policies for audio, scans and student records.
- No permanent behavioural labels from single observations.
- Human approval for consequential academic/developmental decisions.
- Model/provider version recorded for every AI-derived decision.
- Ability to delete/correct derived records while preserving legally required audit history.
- Data minimization: only provide each model the state required for its decision.
- Vendor contracts and data-processing terms must be reviewed before production.

OpenAI's current agent tooling illustrates the general pattern of tracing, guardrails and human approvals; the school should implement equivalent controls regardless of model vendor. ■cite■turn0search1■turn0search7■

23. Observability & Debugging

Every workflow execution should have a trace ID. A trace should show: trigger → state snapshot → model calls → Jev decisions → tool calls → human approvals → state mutation → resulting event.

OpenAI's current Agents SDK tracing records model generations, tool calls, handoffs, guardrails and custom events; this is the level of observability your own platform should aim for irrespective of whether you use that SDK.

■cite■turn0search1■turn0search3■

Build a replay facility: take a historical event and rerun a decision against a new model/spec without changing production state. This is essential for safe improvement.

24. A Day in the Life — Teacher OS

06:45 — System composes today's plan from timetable + curriculum state + unresolved student needs.

08:30 — Teacher opens Today view.

09:00 — Lesson delivered; teacher optionally records quick voice observations.

10:15 — Topic marked complete.

10:16 — Workflow decides which assessment pathway is required.

10:17 — Personalized assessments assembled.

10:20 — Teacher approves batch.

Later — Scans ingested and evaluated.

Evaluation — error patterns classified; low-confidence cases routed to teacher.

End of day — Student states update; intervention tasks generated for students needing them.

Teacher — approves targeted interventions.

Next day — interventions executed; reassessment scheduled automatically where policy says so.

Coordinator — sees class-level exceptions, not hundreds of raw data points.

The teacher's job shifts from content administration and manual bookkeeping toward observation, judgement, relationship and high-quality delivery.

25. What to Build First — Realistic MVP

Do not build the entire vision simultaneously. The first release should prove one complete loop end-to-end.

MVP component	Scope
Skill map	One subject + 1–2 grades
Pedagogy	One topic with validated lesson templates
Question bank	100s of validated items mapped to skills
Assessment	Generate → teacher approve → print
Evaluation	Image/OCR + deterministic grading + Jev error classification
Student State	Skill-level mastery + evidence
Observation	Voice note → structured observations
Intervention	Small validated intervention library
Reassessment	Targeted reassessment
Teacher dashboard	Today + students needing attention
Decision Fabric	Decision Registry + Jev wrapper + evaluation harness
Audit	Trace + event log

If this loop works for one topic, the architecture can expand horizontally across subjects and vertically across grades.

26. 90-Day Build Roadmap

Period	Primary objective
Days 1–15	Core data model, auth, Student State, Knowledge Graph, events, Decision Service, Jev integration
Days 16–30	Curriculum/pedagogy workflow + question bank + assessment assembly
Days 31–45	Assessment ingestion + OCR/vision + evaluation + Jev error decisions
Days 46–60	Mastery engine + observation engine + teacher OS
Days 61–75	Intervention loop + reassessment + daily orchestration
Days 76–90	Evaluation harness, calibration, governance, reporting, pilot hardening

The first production pilot should have a small number of decision classes in advisory mode before any autonomous action.

27. System Success Metrics

Dimension	Metric
Student learning	Mastery growth / skill progression
Personalization	% students receiving differentiated next step
Teacher productivity	Admin minutes saved per teacher/day
Decision quality	Expert agreement / calibration
Assessment quality	Diagnostic value / item validity
Intervention quality	Improvement after intervention
Curriculum quality	Outcome achievement by pedagogy
AI safety	Incorrect autonomous decision rate
Human load	Review rate and review minutes
System reliability	Workflow completion / retry / failure rates
Cost	AI cost per student per month
Latency	Median/p95 decision latency

28. Final Architecture Thesis

The Student OS should become the operating system of the school, not another application used by the school.

The distinction is profound. In a normal school, curriculum, teacher planning, assessment, observations, interventions, reports and handovers are separate human processes connected by spreadsheets, meetings and memory. In this OS they become states and workflows connected by evidence.

The school gradually becomes a closed-loop learning organization: it knows what it intends to teach; knows what was actually taught; knows what each child demonstrated; detects where the child is struggling; selects an intervention; measures whether it worked; learns which pedagogies work; improves curriculum; and carries the resulting knowledge forward to the next teacher and next cohort.

Jev is valuable because the system contains an enormous number of small decisions. It can make those decisions cheap, fast and structured. But the actual institutional intelligence comes from the **combination** of Knowledge Graph + Student State + Evidence + Decision Fabric + Workflow Engine + Human Governance.

Therefore: make the Student State and event architecture permanent; make the workflow contracts explicit; make every AI decision a versioned Decision ID; integrate Jev behind a provider-neutral Decision Service from Day Zero; and let autonomy grow only as evidence justifies it.

That is how this stops being 'an AI school platform' and becomes the **core operating system through which the school actually runs.**

Current references checked 26 September 2026: TypeSafe/System One documentation; OpenAI Agents SDK documentation; LangGraph human-in-the-loop documentation. The final production design should revalidate APIs, pricing, model versions and data-processing terms immediately before implementation.